

Yaw angle time-series generation from video frames using a pre-trained CNN Regressor

Overview

This Jupyter notebook demonstrates how to load a pre-trained Convolutional Neural Network (CNN) regressor model and use it to predict the yaw angle from frames extracted from a video. The process involves splitting the video into frames at regular intervals, predicting the yaw value for each frame, and storing the results in a map structure.

Steps Involved

1. Loading the Pre-trained Regressor Model

The notebook begins by loading a pre-trained CNN regressor model that has been specifically trained to predict the yaw angle from an image of a head.

2. Video Processing

- **Video Splitting:** The video file is split into individual frames. Frames are captured every 0.2 seconds, allowing for consistent sampling across the video.
- **Index Generation:** Each frame is assigned an index corresponding to the time in the video from which it was captured. This index is used to map each frame to its corresponding yaw prediction.

3. Yaw Prediction

For each frame extracted from the video:

- The pre-trained regressor model is used to predict the yaw angle.
- The predicted yaw value and the corresponding index are stored in a map structure.

4. Output

The final output of the notebook is a map where:

- **Key:** The index of the frame (based on the time in the video).
- **Value:** The predicted yaw angle for that frame.

This output map provides a comprehensive view of the yaw angle variations throughout the video, allowing for further analysis or visualization.

Usage

To use this notebook:

1. Ensure that you have the pre-trained CNN regressor model available.
2. Provide the path to the video file you wish to process.
3. Run the notebook to generate the map of yaw predictions.

The map can then be used for various applications, such as detecting head rotations or analyzing the head pose throughout the video.

Import the necessary dependencies

```
In [14]: from tensorflow.keras.models import load_model
import tensorflow as tf # Add this import to make tf available for Lambda layer
import os
from PIL import Image
from tensorflow.keras import backend as K

import subprocess
import os
import sys
import fnmatch

from tensorflow.keras.preprocessing.image import img_to_array, load_img
import numpy as np

import matplotlib.pyplot as plt
```

```
In [11]: # model = load_model('.../trained_models/fixed_attention_cnn.keras')
# Try to load the saved model to confirm it works
# Define named functions for the operations instead of lambda functions
def mean_channel(x, axis=-1, keepdims=True):
    return K.mean(x, axis=axis, keepdims=keepdims)

def max_channel(x, axis=-1, keepdims=True):
    return K.max(x, axis=axis, keepdims=keepdims)

# Register the custom functions
tf.keras.utils.get_custom_objects().update({'mean_channel': mean_channel})
tf.keras.utils.get_custom_objects().update({'max_channel': max_channel})

try:
    model_path = '.../trained_models/fixed_attention_cnn.keras'
    model = tf.keras.models.load_model(model_path, safe_mode=False)
    print("Model loaded successfully!")

    # Make a test prediction
    test_img = x_testing[0].reshape(1, 128, 128, 3)
    prediction = loaded_model.predict(test_img, verbose=0)[0][0]
    actual = y_testing[0]
    print(f"Test prediction: (prediction:.2f)', Actual: (actual:.2f)''")
except Exception as e:
    print(f"Error loading SavedModel: {e}")

Model loaded successfully!
Error loading SavedModel: name 'x_testing' is not defined
```

Add some utility functions for image preprocessing

```
In [8]: def resize_image(image, size=(128, 128)):
    return image.resize(size)

def get_image(directory, filename):
    file_path = os.path.join(directory, filename)

    if os.path.isfile(file_path):
        with Image.open(file_path) as img:
            img = resize_image(img)
            img_array = np.array(img, dtype=np.uint8)
            return img_array

def preprocess_image(image_path, target_size=(128, 128)):
    img = load_img(image_path, target_size=target_size)
    img_array = img_to_array(img) / 255.0
    img_array = img_array.astype(np.float32)
    img_array = np.expand_dims(img_array, axis=0)
    return img_array

def plot_preprocessed_image(preprocessed_image):
    if preprocessed_image.shape[0] == 1:
        img_to_plot = preprocessed_image[0]
    else:
        img_to_plot = preprocessed_image

    plt.imshow(img_to_plot)
    plt.axis('off')
    plt.show()
```

Read the input video and split it into frames

```
In [12]: def extract_frames_ffmpeg(video_path, output_folder, interval=0.2):
    if not os.path.exists(output_folder):
        os.makedirs(output_folder)

    fps = 1 / interval
    command = [
        'ffmpeg',
        '-i', video_path,
        '-vfr', f'fps={fps}',
        os.path.join(output_folder, 'frame_%04d.jpg')
    ]

    subprocess.run(command, check=True)
    print(f"Frames extracted to {output_folder}")

def count_jpg_files(directory):
    count = 0
    for filename in os.listdir(directory):
        if fnmatch.fnmatch(filename, '*.jpg'):
            count += 1
    return count

def list_mov_files(directory):
    return [f for f in os.listdir(directory) if fnmatch.fnmatch(f, '*.mp4')]
```

```
In [34]: # Process a single video
name = "absence"
extension = ".mov"
video_name = f'{name}{extension}'
folder = f'./video/{name}'

video_input_path = f'{folder}/{video_name}'
output_frames_path = folder

# Extract frames if they don't exist
if not os.path.exists(output_frames_path) or count_jpg_files(output_frames_path) == 0:
    extract_frames_ffmpeg(video_input_path, output_frames_path)

# Create predictions map for this video
predictions_map = {}
number_of_frames = count_jpg_files(output_frames_path)
print(f"Total frames extracted: {number_of_frames}")
starting_index = 0.2

# Process each frame
for i in range(1, number_of_frames + 1):
    if (i < 10):
        path = f'{output_frames_path}/frame_000{i}.jpg'
    elif (i < 100):
        path = f'{output_frames_path}/frame_00{i}.jpg'
    else:
        path = f'{output_frames_path}/frame_0{i}.jpg'

    # Check if file exists
    if not os.path.exists(path):
        print(f"Warning: Frame not found at {path}")
        continue

    # Preprocess and predict
    preprocessed_image = preprocess_image(path)
    yaw_prediction = model.predict(preprocessed_image, verbose=0)

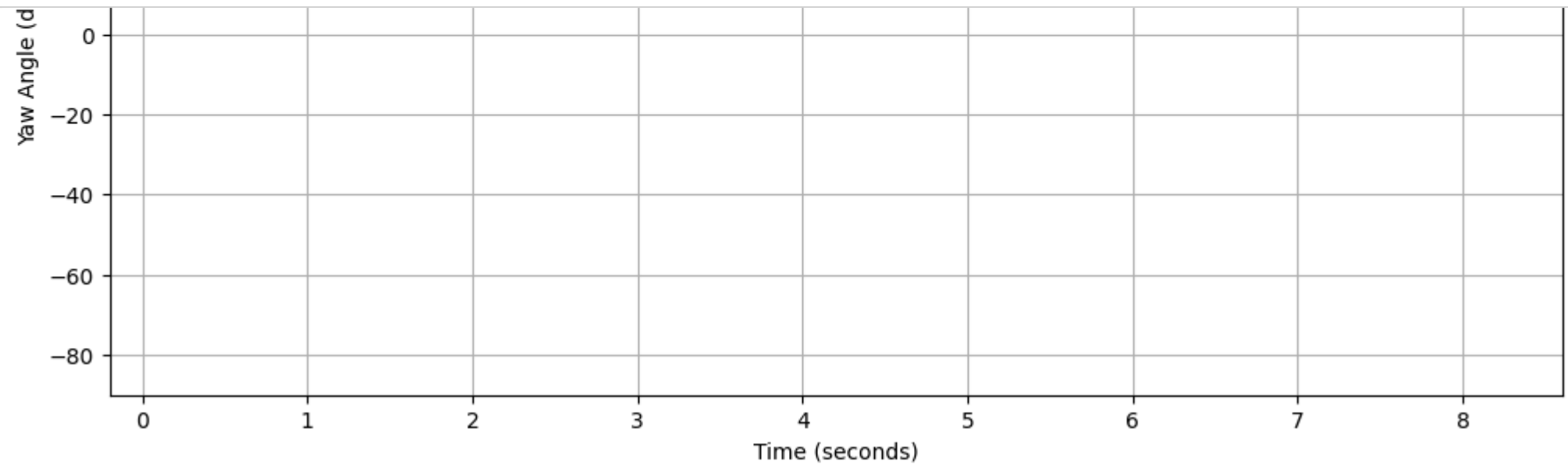
    # Store result
    index = starting_index * i
    predictions_map[index] = yaw_prediction[0][0]

# Plot results
x = list(predictions_map.keys())
y = list(predictions_map.values())

plt.figure(figsize=(12, 6))
plt.plot(x, y, marker='o')
plt.xlabel('Time (seconds)')
plt.ylabel('Yaw Angle (degrees)')
plt.title(f'Yaw Angle Over Time - {video_name_no_extension}')
plt.grid(True)
plt.ylim(-90, 90) # Set Y-axis limits to -90 and 90

# Save plot and data
plt.savefig(f'{folder}/{video_name_no_extension}_yaw_plot.png')
plt.show()

# Save to CSV
predictions_df = pd.DataFrame(list(predictions_map.items()), columns=['Time', 'Yaw'])
csv_path = f'{folder}/{video_name_no_extension}_yaw_data.csv'
predictions_df.to_csv(csv_path, index=False)
print(f"Saved yaw data to {csv_path}")
```



Saved yaw data to ./video/absence/normal_rotation_yaw_data.csv

In [31]: ## Save yaw patterns from all videos to CSV files

```
import pandas as pd

#folder = './video'
video_names = list_mov_files(folder)

for video_name in video_names:
    output_folder = folder

    # Load the frames
    frames_count = count_jpg_files(output_folder)

    if frames_count == 0:
        print(f"No frames found for {video_name}. Skipping...")
        continue

    print(f"Processing {frames_count} frames for {video_name}")

    # Create predictions map
    predictions_map = {}
    starting_index = 0.2

    # Process each frame
    for i in range(1, frames_count + 1):
        if (i < 100):
            path = f'{output_folder}/frame_000{i}.jpg'
        elif (i < 1000):
            path = f'{output_folder}/frame_00{i}.jpg'
        else:
            path = f'{output_folder}/frame_0{i}.jpg'

        # Check if file exists
        if not os.path.exists(path):
            print(f"Warning: Frame not found at {path}")
            continue

        # Preprocess and predict
        preprocessed_image = preprocess_image(path)
        yaw_prediction = model.predict(preprocessed_image, verbose=0)

        # Store result
        time_point = starting_index + i
        predictions_map[time_point] = yaw_prediction[0][0]

    # Save to CSV
    predictions_df = pd.DataFrame(list(predictions_map.items()), columns=['Time', 'Yaw'])
    csv_path = f'{folder}/{video_name_no_extension}_yaw_data.csv'
    predictions_df.to_csv(csv_path, index=False)
    print(f"Saved yaw data to {csv_path}")

print("All video yaw patterns saved to CSV files.")
```

All video yaw patterns saved to CSV files.

In [12]:

```
x = list(predictions_map.keys())
y = list(predictions_map.values())

print(predictions_map)

plt.plot(x, y, marker='o')
plt.xlabel('Time')
plt.ylabel('Yaw value')
plt.title('Yaw values across the time')
plt.grid(True)
plt.ylim(-90, 90) # Set Y-axis limits to -90 and 90
plt.show()
```

{0.2: -22.149895, 0.4: -23.401308, 0.6000000000000001: -24.494002, 0.8: -24.766348, 1.0: -24.996485, 1.2000000000000002: -24.87633, 1.4000000000000001: -24.304657, 1.6: -24.274889, 1.8: -36.94259, 2.0: -30.16348, 2.2: -32.04467, 2.4000000000000004: -35.724483, 2.6: -27.80121, 2.8000000000000003: -21.90367, 3.0: -28.004562, 3.2: 29.046644, 3.4000000000000004: 37.72852, 3.6: 35.611225, 3.8000000000000003: 33.944767, 4.0: -17.551815, 4.2: -32.152256, 4.4: -33.3654, 4.6000000000000005: -36.15797, 4.800000000000001: -36.50573, 5.0: -30.35959, 5.2: -29.235329, 5.4: -22.994583, 5.6000000000000005: 5.257295, 5.800000000000001: 44.314487, 6.0: 40.062973, 6.2: 29.226763, 6.4: -22.617603, 6.6000000000000005: -23.783236, 6.800000000000001: -25.269875, 7.0: -26.099443, 7.2: -25.725424, 7.4: -23.40471, 7.6000000000000005: -23.281487}

